

Chapter 58: Design a Web Crawler

Personal Notes

A web crawler visits websites and downloads their contents. It's how Google, Bing, and every search engine collect the pages they later index. It's also how many other systems bootstrap themselves: monitoring services, plagiarism checkers, price aggregators, archival projects (Wayback Machine), and countless data-collection pipelines. Behind the simple idea ("fetch URLs, extract links, fetch those too") lies a genuinely challenging distributed system.

This chapter designs a web-scale crawler capable of fetching billions of pages. Along the way you'll meet the URL FRONTIER (the ranked queue of what to crawl next), the DEDUPLICATION challenge at internet scale (Bloom filters!), the POLITENESS problem (don't hammer any single website), DNS at scale, and the crawl-store-parse pipeline. Beyond crawlers, these patterns power any large batch pipeline: monitoring, migration, indexing. Master this and you understand how the internet gets discovered.

1. The Problem

Design a system that starts with a set of SEED URLs, fetches each page, extracts links from it, and recursively fetches those linked pages. Store the fetched HTML for downstream indexing/analysis. Continuously crawl to keep the index fresh (revisit pages that change). Respect politeness rules — don't attack anyone's server. Scale to billions of pages.

```
Seeds: [example.com, wikipedia.org, cnn.com, ...]
```



```
Fetch each URL → HTML
```



```
Parse HTML → extract more URLs
```



```
Add new URLs to the queue
```



```
Loop... billions of times
```

```
Real-world scale (Google Search):
```

```
Index size:    ~100 billion pages
```

```
Fetch rate:    ~50 GB / second globally
```

```
Refresh cycle: days (fresh sites) to months (static sites)
```

2. Step 1 — Requirements Gathering

2.1 Functional Requirements

- Given SEED URLs, crawl all reachable pages
- Extract links from HTML and enqueue new URLs
- Store fetched HTML for downstream use (indexing, analysis)
- Deduplicate — don't crawl the same URL twice unnecessarily
- Respect robots.txt (site-specific crawl rules)
- Re-crawl pages periodically to detect updates
- (Optional) Prioritize important pages, handle media (images, PDFs), obey noindex tags

2.2 Non-Functional Requirements

- Scale — target 1B pages/month = ~400 pages/second sustained (peak up to 10K/sec)
- Politeness — no domain gets more than N requests/sec from us
- Robustness — one bad page can't crash a worker; retry with backoff
- Freshness — high-priority sites re-crawled hourly; long-tail monthly
- Storage — ~30 KB per HTML page (compressed); billions of pages = petabytes

2.3 Constraints to Confirm

- Text only, or also media (PDFs, images, videos)?
- HTTP/HTTPS only, or also FTP, Gopher, etc.?
- Follow redirects up to what depth?
- Store the raw HTML, or a parsed/normalized form? Both?

The critical clarification: "Is this a general web crawler (Googlebot style) or a targeted crawler (specific site or vertical)?" A general crawler must handle every kind of website. A targeted crawler can hard-code assumptions about the sites it visits — much simpler. Interviewers usually mean general web crawler. Confirm this upfront.

3. Step 2 — Capacity Estimation

Fetch Rate

Target: 1B pages/month
 $1\text{B} \div 30 \text{ days} \div 86,400 \text{ seconds} \approx 400 \text{ pages/sec average}$
 $\times 3 \text{ for peak} \qquad \qquad \qquad \approx 1,200 \text{ pages/sec peak}$

Each fetch:
1× DNS lookup ($\sim 10 \text{ ms avg}$, cached mostly)
1× TCP + TLS setup ($\sim 100 \text{ ms first time}$, $\sim 5 \text{ ms}$ if pooled)

1× HTTP GET (~200 ms average, up to seconds)

Total time per fetch: ~300 ms average

Concurrent connections needed: $1200 \times 0.3 = 360$ workers active

Realistic: 500-1000 workers to handle latency variance

Storage

Average HTML page (compressed): ~30 KB

1B pages $\times$ 30 KB = 30 TB per crawl cycle

If retained for N cycles (for diffing / freshness detection): ~300 TB

Media (images, PDFs): 10× larger – 300 TB per cycle

Total: ~1 PB scale storage. Object store (S3, GCS) is the answer.

URL Frontier Storage

Discovered URLs (seen at least once): 100B+ URLs

Each URL: ~80 bytes $\rightarrow$ 8 TB total

Not-yet-crawled URLs (in the frontier): 1-10B URLs

In-memory portion (hot): 10 GB

Disk-backed remainder: TB range

Bloom filter for dedup: 100B URLs $\times$ 10 bits $\approx$ 125 GB in RAM

(Distributed across many machines)

4. Step 3 — Data Model

Table: url_metadata ← One row per known URL
(many billions)

url_hash (PK)		url		last_crawled_at		status		priority
				content_hash		next_crawl_at		domain

Sharded by url_hash.

Table: crawl_history ← Log of every crawl attempt

url_hash		crawl_id		timestamp		http_status		latency_ms		content_size
----------	--	----------	--	-----------	--	-------------	--	------------	--	--------------

Table: page_content (in blob store)

Path: s3://crawler/content/{yyyymmdd}/{shard}/{url_hash}.html.gz

→ Raw HTML, compressed, deduplicated by content_hash

Table: domain_state (Redis)

Key: "domain:{example.com}"

Value: {

last_fetch_time: <timestamp>,

```
    crawl_delay_ms: 1000,                ← from robots.txt
    rate_limit_per_sec: 5,
    active_workers: 2
}
```

URL Frontier (persistent queue):
Kafka topic partitioned by domain hash
Multiple queues (priority tiers)

5. The Core Challenge — The URL Frontier

The URL FRONTIER is the queue of URLs waiting to be crawled. It sounds simple — a FIFO queue — but at web scale, its design is the single most important choice in the whole system. A great frontier gets these four things right:

1. PRIORITIZATION — high-value pages first (popular, fresh, high PageRank)
2. POLITENESS — never overload any single domain, regardless of how many URLs you have for it
3. FRESHNESS — periodically requeue pages to detect updates
4. PERSISTENCE — survive crashes; billions of URLs won't fit in memory

Naive Approach (Fails)

One giant FIFO queue of all URLs.
Workers pull, fetch, discover new URLs, add to queue.

Problem: 10,000 URLs from wikipedia.org enter the queue at once.
Workers slam wikipedia.org with 10,000 requests → we get IP-banned.

Politeness DEMANDS per-domain rate limiting.

Mercator-Style Frontier (The Interview Answer)

Named after the Mercator crawler paper (1999). Split the frontier into TWO layers:

LAYER 1 — Priority Queues (F queues, $F \approx$ number of priority tiers)
F1: HIGH priority (Wikipedia, news sites, recently changed)
F2: MEDIUM priority
F3: LOW priority (long-tail static sites)

LAYER 2 — Per-Domain FIFOs (B queues, $B \approx$ number of workers)
Each worker OWNS one queue.
Each queue contains URLs from ONE DOMAIN at a time.
When empty, worker requests a new URL from priority queues.
A "gap" timestamp per queue enforces politeness.

Flow:

Worker: "give me the next URL"
→ Frontier: "next URL is /page from example.com. Domain
was fetched 800 ms ago; wait 200 ms then fetch."

Politeness is enforced by DESIGN — worker literally can't fetch a domain faster than the gap allows.

The clean invariant: In a Mercator frontier, EACH WORKER handles EXACTLY ONE DOMAIN at a time. Politeness is a consequence of the architecture, not a runtime check. Multiple workers CAN fetch different pages from the same domain in parallel by routing through the same worker's queue — but a worker owning that domain naturally serializes requests.

6. Deep Dive: Deduplication at Scale

Before fetching a URL, check: have we seen it before? At 100 billion known URLs, this is not a trivial hash-set operation. A naive set uses $\sim 100 \text{ bytes per URL} \times 100\text{B} = 10 \text{ TB}$ in memory. Even distributed, that's expensive.

Bloom Filters — The Standard Answer

A Bloom filter is a probabilistic data structure that answers "have I seen X?" with either "definitely no" or "probably yes." Never gives "definitely yes" — but false positives are BENIGN for a crawler (we skip a URL we thought we'd seen; slight loss but no correctness issue).

Bloom filter for 100B URLs at 1% false-positive rate:

Bits per element: ~ 10

Total bits: $100\text{B} \times 10 = 1 \text{ trillion bits} = 125 \text{ GB}$

Distribute across 16 shards (by url_hash):

Each shard: $\sim 8 \text{ GB}$ Bloom filter → fits in RAM easily

Check flow:

1. $\text{hash}(\text{url}) \rightarrow \text{shard_id}$
2. Query shard's Bloom filter → "probably seen" or "definitely not"
3. If "definitely not" → new URL, add to frontier
4. If "probably seen" → confirm with authoritative store (rare)

Cost:

100 KB/sec URL check throughput per shard.

16 shards $\times$ 100 KB/sec = plenty of headroom.

Bloom filters can't remove elements: Once an element is added, you can't take it back. Every year or so, rebuild the Bloom filter from the authoritative URL store, dropping URLs older than N months. Counting Bloom filters allow deletion but use more space — usually not worth it for crawlers.

URL Normalization Before Hashing

Same URL can appear in different forms — must normalize before dedup:

- `http://example.com` and `http://example.com/` (trailing slash)
- `http://EXAMPLE.com` and `http://example.com` (case)

- `http://example.com/page?a=1&b=2` and `?b=2&a=1` (query param order)
- `http://example.com#section` vs `http://example.com` (fragment)
- `http://example.com/../foo` (path normalization)

```
String normalize(String url) {
    URI u = URI.create(url);
    String scheme = u.getScheme().toLowerCase();
    String host = u.getHost().toLowerCase();
    // Sort query params, strip fragment, remove default ports, ...
    return scheme + "://" + host + normalizePath(u.getPath()) +
        sortQuery(u.getQuery());
}
```

7. Deep Dive: Politeness

"Politeness" is what separates a helpful crawler from a DDoS attack. Google is polite; without it, they'd have been banned by every website. Politeness has THREE components:

7.1 robots.txt

A file at `/robots.txt` on every domain that tells crawlers what NOT to visit. Fetch and parse this BEFORE crawling any URL on the domain. Cache aggressively (24-hour TTL is standard).

```
User-agent: *
Disallow: /admin/          # No crawler should fetch these
Disallow: /private/
Crawl-delay: 5             # Wait 5 seconds between requests
Sitemap: https://example.com/sitemap.xml

User-agent: MyCrawler
Allow: /public/
Disallow: /                # But everything else is off-limits
```

7.2 Per-Domain Rate Limiting

Even if robots.txt allows crawling, don't hammer the domain. Default: max 1 request per second per domain. Small sites shouldn't get 100 QPS from us. Frontier design enforces this naturally (one worker per domain).

7.3 User-Agent Identification

```
User-Agent: MyCrawler/1.0 (+https://example.com/crawler-info)

↑ tells site owners:
- Who is crawling
- Contact info if problems arise
- Legitimate identification (not spoofed)
```

Being polite is legally + ethically necessary: Aggressive crawling can trigger CFAA violations in the US, GDPR issues in the EU, and denial-of-service lawsuits everywhere. Even Google was sued repeatedly in its early days. Get politeness right by default — respect robots.txt, throttle per domain, identify yourself, honor Crawl-Delay directives.

8. Deep Dive: DNS at Scale

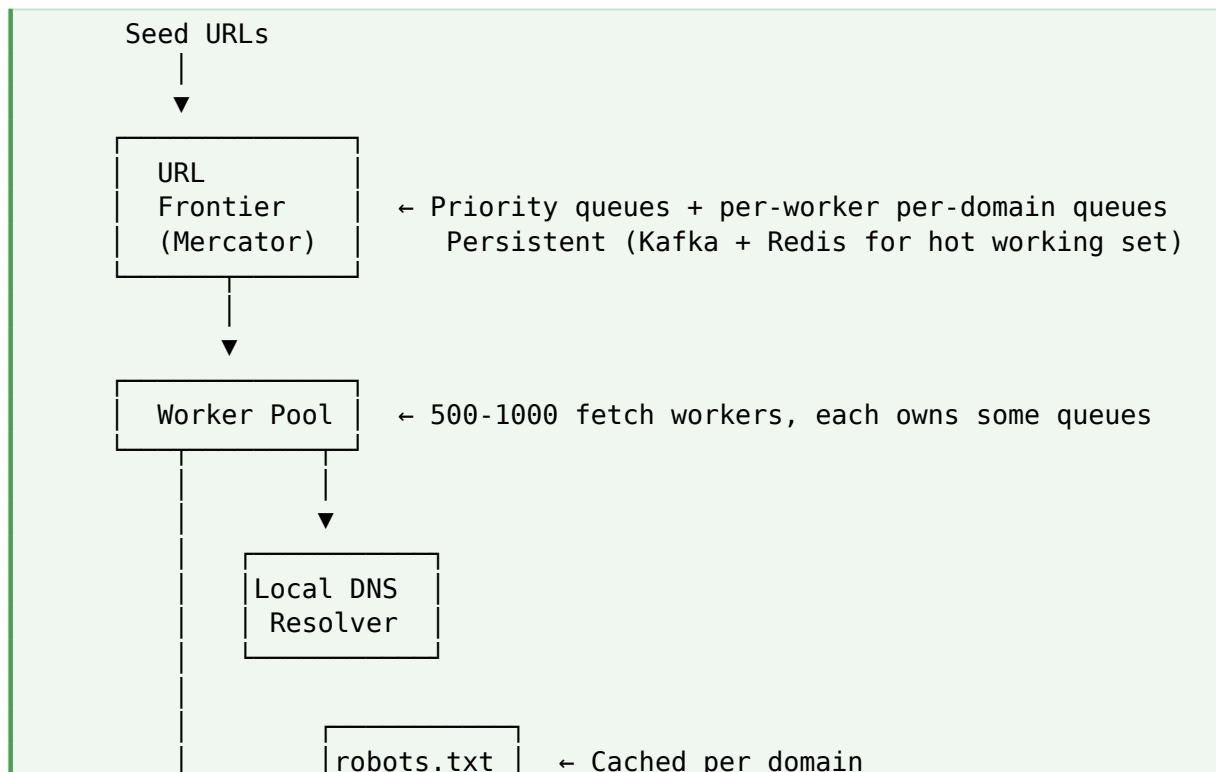
Every URL needs DNS resolution to convert hostname to IP. At 1200 fetches/sec, that's 1200 DNS lookups/sec. Public DNS resolvers won't tolerate that — you'll get rate limited within minutes.

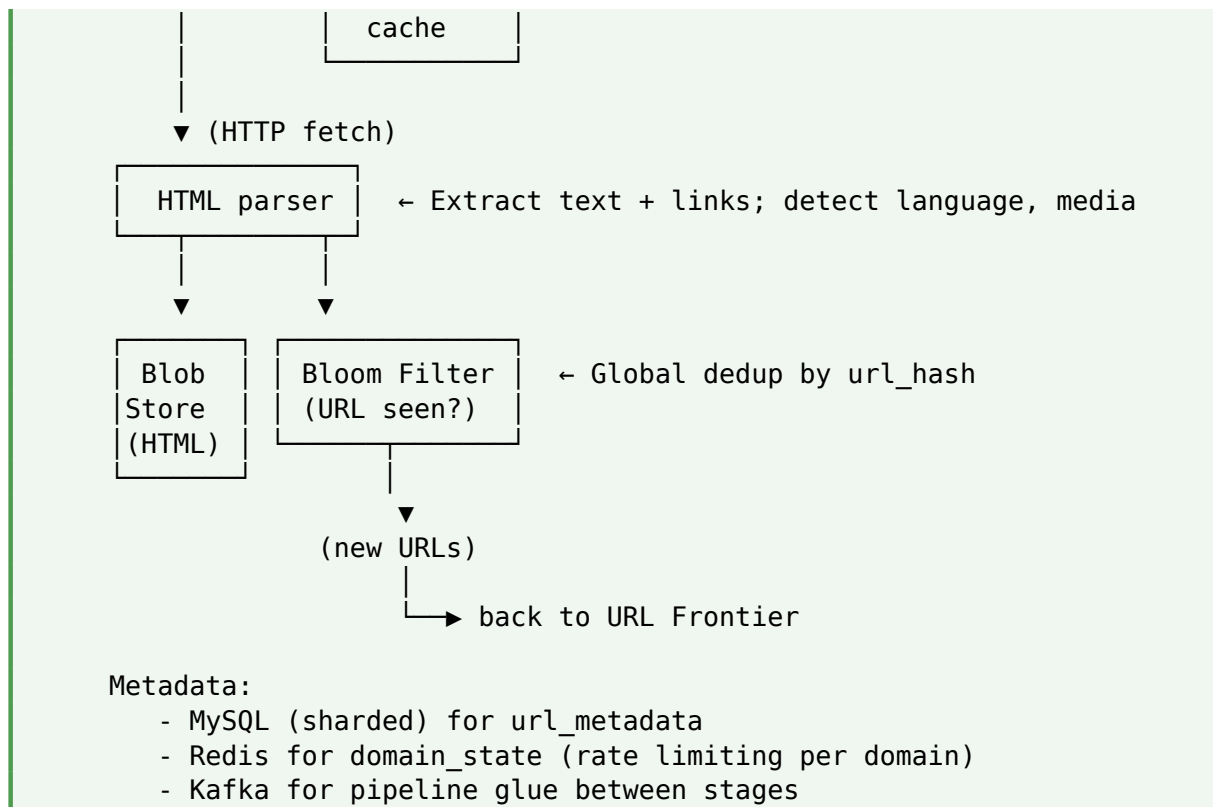
Solution — Local DNS Cache + Resolver

- Run your own local DNS resolver (dnsmasq, unbound) on every worker
- Cache DNS responses respecting TTL (typically 5 min to 24 hours)
- Prefetch DNS for hot domains — Google, Wikipedia, etc.: keep entries always fresh
- Ask multiple upstream DNS servers in parallel for reliability

The subtle DNS bottleneck: Java's `InetAddress.getByName()` historically CACHED DNS forever (until JVM restart), which broke long-running crawlers. Modern JVMs use `networkaddress.cache.ttl`; check this configuration. For serious scale, bypass the JVM's resolver entirely with a native DNS client library.

9. Step 4 — High-Level Architecture





Service Roles

- URL Frontier — persistent priority queue + per-worker per-domain sub-queues
- Fetch Workers — pull from frontier, fetch HTTP, dispatch to parser
- HTML Parser — extract text + outbound links
- Dedup Service — Bloom filter cluster + authoritative URL store
- robots.txt Cache — per-domain rules with TTL
- Blob Store — raw HTML, keyed by url_hash + crawl date
- Metadata Service — url state (last crawled, priority, hash, etc.)

10. Deep Dive: Content Parsing & Storage

HTML Parsing Steps

5. Fetch raw HTML
6. Compute content_hash (SHA-256) — detects if page changed since last fetch
7. Parse HTML into DOM (libraries: jsoup, lxml, gumbo)
8. Extract outbound links from <a href> tags
9. Extract main text content (strip nav, ads, boilerplate)
10. Detect language for language-specific indexing
11. Store to blob store; enqueue new URLs after normalization

Content Deduplication

Two DIFFERENT URLs can serve IDENTICAL content (e.g., mirror sites, CDN copies, printer-friendly versions). Storing both wastes 50%+ of storage. Compute `content_hash`; if a hash already exists, store only a `POINTER` + treat as an alias.

```
Fetch page → compute SHA-256(body) → content_hash
```

```
Check store: does content_hash exist?
```

```
  YES → this page is a duplicate. Store metadata but reference existing blob.
```

```
  NO  → store blob; add hash to index.
```

```
Result: crawled internet contains ~30-40% duplicate content.
```

```
  This dedup halves your storage bill.
```

Storage Format

- Compress with gzip or brotli — HTML compresses ~4-6× typically
- Batch small pages into WARC files (Web ARChive format — industry standard)
- Tiered storage: hot pages in S3 Standard, cold in S3 Glacier/Deep Archive

11. Deep Dive: Freshness — Recrawling

The web changes. News sites update hourly; static docs may not change in years. Naively recrawling everything on a fixed cycle wastes 80% of fetches. Priority-based recrawling captures the important changes without waste.

Change Detection

- Store `content_hash` from each crawl
- On recrawl: if new hash matches old, no update — increase interval, deprioritize
- If hash differs: mark page as changed, decrease interval, prioritize

Adaptive Freshness

Tier	Example	Recrawl Interval
High priority	Front pages of major news sites, homepages	1-2 hours
Medium	Blog posts, product pages	1-3 days
Low	Static reference pages, archives	Weeks to months
Dead	404/5xx repeatedly	Give up after retries

The sitemap advantage: Well-run websites publish sitemap.xml files listing all their pages, sometimes with <lastmod> timestamps. Trust these hints! A news site's sitemap tells you "this article changed at 09:00" — skip pages that haven't changed. Massively reduces useless fetches.

12. Deep Dive: Handling Failures & Retries

Error	Cause	Action
Connection refused	Server down or blocking us	Retry with exponential backoff, up to 3 attempts
DNS failure	Domain doesn't exist or DNS issue	Retry after 1 hour, then mark dead
HTTP 429 / 503	Rate limited — we're being polite too little	Extend crawl delay for this domain
HTTP 5xx	Server error, may be transient	Retry with backoff, count against domain reputation
HTTP 4xx	Page doesn't exist / forbidden	Give up on this URL, mark 404
Redirect loop	Site config bug	Cap at 5 redirects, then error
Timeout	Slow server or huge page	Timeout at 30s; retry with backoff
Content too large	Attack or CDN misconfiguration	Cap fetch size at ~10 MB; log + skip

Crawler traps are real: Some websites generate INFINITE URLs (calendar sites with year=1999&year=2000&... , dynamic filters that create new URLs on every parameter combo). Detect: if a domain contributes 1M+ URLs and none produce unique content, throttle heavily or blacklist. Google's Googlebot has extensive trap detection.

13. Additional Features

13.1 Distributed Crawling

Scale beyond one datacenter. Partition domains by hash — each region owns a subset. Advantages: latency wins (fetch European sites from Europe), regulatory compliance (GDPR data locality). Complication: keeping the global URL Bloom filter consistent.

13.2 JavaScript-Rendered Sites

Modern sites render content client-side with React/Vue/Angular. A simple HTTP fetch returns an empty shell. Need headless browsers (Puppeteer, Playwright) to run JavaScript. 100× slower per fetch. Google runs a dedicated "rendering service" for these; small crawlers skip JS-heavy sites.

13.3 Media Handling

Images, videos, PDFs, audio. Same fetch mechanism but different parsing: OCR for images, thumbnail extraction for videos, PDF-to-text for documents. Media is 10× the size of HTML — often crawled by a separate media crawler with different bandwidth budgets.

13.4 Priority & PageRank

Not all pages equal — CNN homepage is more important than a random forum thread. Use INCOMING LINK COUNT as a quick proxy for importance; more sophisticated crawlers compute PageRank offline and feed priority scores back to the frontier.

13.5 Detecting Spam & Malicious Content

Content farms, spam sites, malware distributors. Combine signals: domain reputation, content classifiers, page similarity, ad density. Skip or deprioritize known-bad domains. Google has entire teams devoted to this.

14. Failure Modes

Failure	Mitigation
Frontier node crash	Kafka persistence means URLs aren't lost; new node picks up from last commit
Worker crash mid-fetch	URL re-queued after timeout; may cause 1 duplicate fetch (acceptable)
Bloom filter corruption	Rebuild from url_metadata store; overshoot with false-positives briefly
Target site DDoS-blocks us	Detect 403/429 spike → back off aggressively, add domain to blacklist
Storage backpressure	S3 slow → workers back off; frontier absorbs slack in Kafka
Politeness bug (too fast)	Get IP-banned by target site → rotate outbound IPs; audit domain rate limits
Runaway domain (crawler trap)	Detect via URL count per domain vs unique content ratio → throttle

15. Trade-offs Summary

Decision	Trade-off
BFS vs DFS traversal	BFS = broader coverage first; DFS = depth-first (bad for the web)
One giant frontier vs sharded	Giant = simpler; sharded = scales but must coordinate politeness

Decision	Trade-off
Bloom filter vs exact set	Bloom = 100× less memory + 1% false positives; exact = perfect but 100× RAM
Recrawl everything vs adaptive	Everything = simple; adaptive = 5× fewer fetches for same freshness
Store raw HTML vs parsed	Raw = flexible for later; parsed = smaller but locks in schema
Retry aggressively vs give up	Aggressive = catches transient failures; wastes work on dead pages
Sync fetching vs async	Sync = simpler; async = 10× throughput per worker
Follow all links vs filtered	All = comprehensive; filtered = avoids traps + spam earlier

16. Common Mistakes

Mistake 1: Ignoring politeness

Every candidate wants to talk about scale. Politeness is what separates a real crawler from a DDoS tool. Bring up robots.txt, per-domain rate limits, and User-Agent identification unprompted.

Mistake 2: One flat URL queue

A single FIFO destroys politeness — 10,000 URLs from one domain would be crawled in rapid succession. Mercator-style per-domain queues owned by workers is the standard answer.

Mistake 3: Using an in-memory HashSet for dedup

At 100B URLs, that's 10 TB of RAM. Bloom filters solve this at ~125 GB with 1% false-positive rate. False positives are BENIGN (skip a URL you thought you'd seen) so this trade-off is essentially free.

Mistake 4: Forgetting URL normalization

http://example.com and http://Example.com/ and http://example.com?a=1&b=2 vs ?b=2&a=1 — they're the same page but three different strings. Normalize FIRST, then hash for dedup.

Mistake 5: No DNS caching

Public DNS resolvers will rate-limit you at 1200 lookups/sec. Run local DNS caches on every worker; respect TTL. This is a common gotcha in real crawler operations.

Mistake 6: Recrawling everything on the same cycle

Wikipedia's front page changes hourly; a 2005 blog post never changes. Track content_hash and adapt: recrawl frequently-changing pages often, static pages rarely. Cuts fetch load by 5-10×.

Mistake 7: Ignoring crawler traps

Some sites accidentally (or maliciously) generate infinite URLs. Without detection, one crawler trap can consume all your capacity. Monitor URL-count-per-domain vs unique-content ratio; throttle or blacklist offenders.

17. Best Practices

- Use Mercator-style frontier — priority queues + per-worker per-domain FIFO queues
- Bloom filter for URL dedup — 125 GB for 100B URLs at 1% false positives
- Normalize URLs BEFORE hashing for dedup (case, params, trailing slashes)
- Respect robots.txt religiously — fetch once per 24 hours per domain, cache
- Cap per-domain rate at 1 req/sec by default; respect Crawl-Delay directives
- Run local DNS resolver per worker; respect TTLs
- Track content_hash to detect page changes; adapt recrawl frequency
- Trust sitemap.xml hints when available — often lists lastmod timestamps
- Cap fetch size (~10 MB), redirects (~5), and timeouts (~30s)
- Detect crawler traps via URL-count-per-domain vs unique-content ratio

18. Quick Reference

Concept	Meaning
URL Frontier	Persistent queue of URLs to crawl next
Mercator design	Priority queues + per-worker per-domain FIFO
Politeness	Rate-limiting per domain, robots.txt, User-Agent ID
robots.txt	Per-domain crawler rules; cache 24h per domain
Crawl-Delay	Seconds between requests, honored by polite crawlers
Bloom filter	Probabilistic set for O(1) 'seen this?' with tiny RAM
URL normalization	Case, params, trailing slash, fragment stripping
content_hash	SHA-256 of body; detects unchanged pages + duplicates
WARC format	Web ARChive — standard for storing crawled data
Crawler trap	Site with infinite URLs (calendars, dynamic filters)
Adaptive freshness	Recrawl frequently-changing pages more often
Sitemap.xml	Site-published list of URLs with lastmod hints

Concept	Meaning
Content dedup	Two URLs same body → single blob, alias pointer

19. Related Interview Problems

Crawler patterns generalize to many batch-oriented systems:

12. Design a Search Engine — crawler feeds indexer feeds ranker
13. Design a Price Comparison Aggregator — targeted crawler over e-commerce sites
14. Design a News Aggregator — RSS/sitemap-driven crawler with freshness priority
15. Design a Web Archive (Wayback Machine) — crawler + tiered storage for versioning
16. Design a Site Monitoring Service — targeted crawler + alerting
17. Design a Broken Link Checker — crawler restricted to a specific site's graph
18. Design a Distributed Job Scheduler — same URL frontier patterns for arbitrary jobs
19. Design a Large-Scale Data Migration Pipeline — same producer/consumer + politeness patterns

20. Final Takeaway

Web crawlers seem simple but are among the trickiest distributed systems to build right. Get the URL Frontier wrong and you either DDoS the internet or crawl inefficiently. Get dedup wrong and you re-crawl the same content endlessly. Get politeness wrong and you get IP-banned. Get freshness wrong and your index goes stale within days. The Mercator design solves the frontier problem elegantly; Bloom filters solve dedup at scale; adaptive freshness solves recrawl efficiency. These three ideas are the essence.

For interviews, walk it top-down: seeds → frontier (Mercator with priority queues + per-domain sub-queues) → fetch workers (DNS cache, robots.txt check, per-domain rate limit) → parse (extract links, dedup via Bloom, store body) → recrawl priority based on content_hash change frequency. Then be ready for deep-dives: "What if a domain has 1B pages? How do you handle JavaScript sites? What if a page 500s repeatedly?" These questions probe your understanding of the operational realities.

Key Takeaway: Web crawler = URL FRONTIER + POLITENESS + DEDUP. Use MERCATOR frontier: priority queues + per-worker per-domain FIFO queues (politeness by design). BLOOM FILTER for URL dedup at 125GB for 100B URLs. NORMALIZE URLs before hashing (case, params, trailing slash). Respect robots.txt + Crawl-Delay religiously. Local DNS resolver per worker. Content_hash-based ADAPTIVE FRESHNESS — recrawl changing pages more often. Detect and throttle CRAWLER TRAPS. Cap fetch size, redirects, timeouts. Content-hash dedup halves storage.

21. What's Next?

Web crawler done. Remaining designs and deep dives:

- Design a Search Autocomplete — trie-based (Ch 36!), prefix caching, popularity
- Design a Search Engine — indexer + ranker + query serving
- Design a Notification System — cross-channel delivery, priorities, batching
- Design a Payment System — idempotency, sagas, PCI compliance
- Design a Metrics/Monitoring System (Datadog-style) — time-series data
- Design a Distributed Job Scheduler — Cron at scale, resilient task queues
- Deep Dive: Databases Internals — B-trees, LSM trees, WAL, MVCC
- Deep Dive: Kafka Architecture — partitions, consumer groups, exactly-once
- Deep Dive: Consensus Algorithms — Raft (worked example), Paxos, leader election
- Deep Dive: Distributed Transactions — 2PC, sagas, event sourcing